



Diseño de Compiladores I

Compiladores e Intérpretes



Análisis Sintáctico
Yacc

Análisis Sintáctico:

Estrategias de Parsing

- ▶ Agrupa los tokens del programa fuente en frases gramaticales que el compilador usará en las siguientes etapas.
- ▶ Obtiene una cadena de tokens del Analizador Léxico, y verifica que la cadena de tokens pueda generarse mediante la gramática del lenguaje.
- ▶ Parsing Ascendente (bottom up)
 - ▶ Construye el árbol desde las hojas a la raíz
- ▶ Parsing Descendente (top down)
 - ▶ Construye el árbol desde la raíz a las hojas



Gramáticas LR

- ▶ Parsing Ascendente LR(0) y SLR(1) (LR(1) Simplificado)
 - ▶ Intentan **predecir** qué reducciones aplicar sin necesidad de ver toda la entrada
 - ▶ Parsers shift/reduce
 - ▶ Conflictos
 - ▶ LR(0) → Falla
 - ▶ SLR(1) → Analiza un token más para decidir qué acción tomar
- ▶ LALR(1) (Lookahead LR(1))
 - ▶ Usan reglas más precisas que SLR(1) para resolver conflictos.
 - ▶ **YACC usa esta técnica**

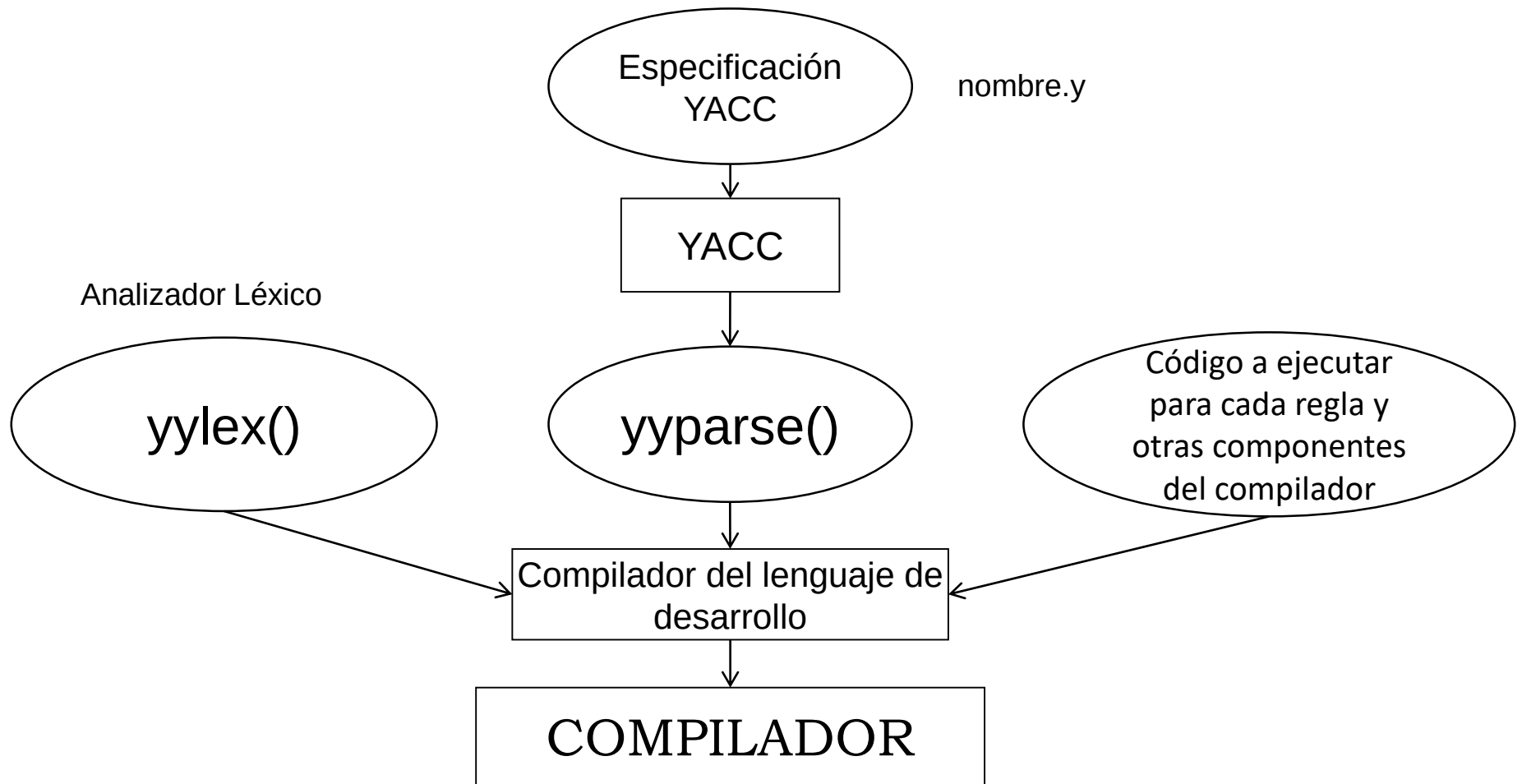
YACC

Yet Another Compiler Compiler

YACC

- ▶ YACC provee una herramienta general para analizar estructuralmente una entrada.
- ▶ Requiere una especificación que incluye:
 - ▶ Un conjunto de reglas que describen los elementos de la entrada (Gramática)
 - ▶ Un código a ser invocado cuando una regla es reconocida
 - ▶ Un Analizador Léxico que se encargue de proveer tokens

Yacc



¿Cómo se usa YACC?

- ▶ Escribir una especificación YACC conteniendo la gramática. (.y por convención)
 - ▶ Ejemplo: `gramatica.y`
- ▶ Escribir un Analizador Léxico. El método o función léxica debe ser:
 - ▶ **`int yylex()`**

Nota: `yyparse` invoca a `yylex` cada vez que necesita un token.

- ▶ Ejecutar YACC con la especificación como parámetro, para generar el código fuente del parser.

Byacc para Java: `yacc -J gramatica.y`

La salida es un archivo llamado **`Parser.java`** conteniendo el método `yyparse()`

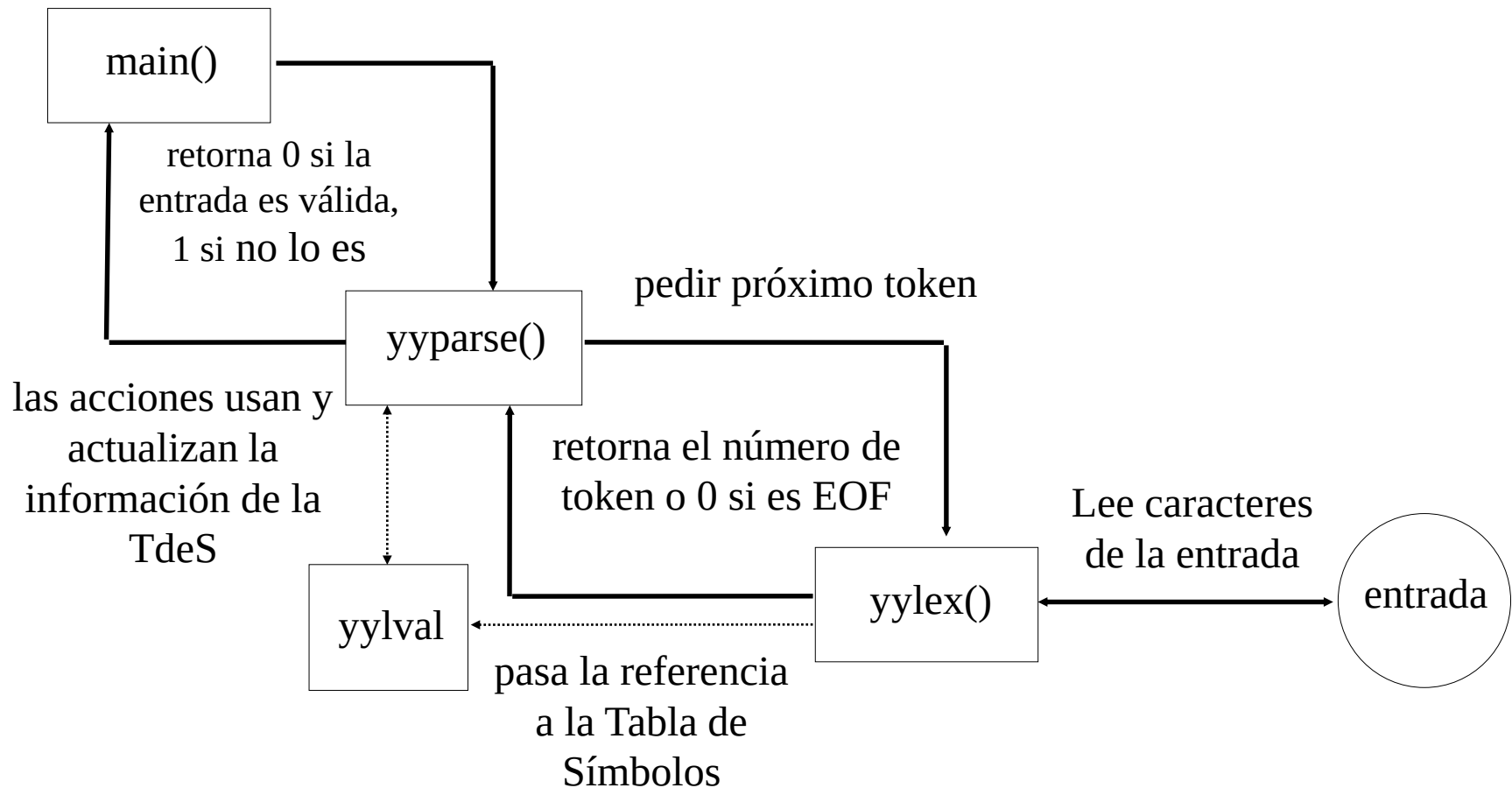
Byacc para C: `byacc gramatica.y`

La salida es un archivo llamado **`y_tab.c`** conteniendo la función `yyparse()`.

- ▶ Compilar y vincular los fuentes del Analizador Sintáctico, Analizador Léxico y todo otro código creado.



Interacción entre yyparse y el Analizador Léxico



Especificación YACC

Formato:

declaraciones

%%

reglas gramaticales

%%

código

Tokens

- ▶ El Analizador Léxico detecta un token y retorna un número de token al parser.
- ▶ El número de token le permite al parser identificar el token.
- ▶ Los números de token son definidos por YACC cuando procesa los tokens declarados en la especificación.
- ▶ Cada carácter ASCII es definido como un token cuyo número es su valor ASCII (0 a 256).
- ▶ Los tokens definidos por el usuario comienzan en 257.

Tokens

- En las declaraciones de la especificación, se debe incluir la lista de tokens:

```
%{  
    import java.io.*;  
    import Lex.Lex;  
}%  
  
%token ID CTE ASIG IF THEN ELSE WHILE  
  
%%  
prog          : asignacion  
               ;  
asignacion    : ID ASIG expresion  
               ;
```

Tokens

- ▶ Si, en la especificación para YACC, se incluye:

%token ID CTE ...

- ▶ En **byacc** para Java, en el archivo Parser.java, junto con yyparse, se generan:

public final static short ID=257;

public final static short CTE=258;

...

```
public final static short ID=257;  
public final static short CTE=258;  
public final static short ASIG=259;  
public final static short IF=260;  
public final static short THEN=261;  
public final static short ELSE=262;  
public final static short WHILE=263;
```

Tokens

- ▶ Si, en la especificación para YACC, se incluye:

`%token ID CTE ...`

- ▶ En **yacc** para C, se generan sentencias `#define` para definir los números de tokens:

`#define ID 257`

`#define CTE 258`

`...`

- ▶ Estas definiciones son ubicadas en el archivo `y_tab.c`, junto con la función **yyparse**, o pueden generarse en un archivo separado, llamado `y_tab.h`.

- ▶ Para ello, se debe ejecutar `byacc -d gramatica.y`



Tokens – **yylval**

- ▶ Para pasar información del token al parser, se utiliza una variable externa llamada **yylval**.

Especificación YACC

Formato:

declaraciones

%%

reglas gramaticales

%%

código

Especificación YACC

Gramática:

- ▶ Tokens, que son un conjunto de símbolos terminales
- ▶ Elementos sintácticos, que son un conjunto de símbolos no terminales
- ▶ Reglas de producción
- ▶ Una regla **start** que reduce todos los elementos de la gramática a una sola regla.

Especificación YACC

Sección de Reglas

► Formato:

no terminal : definición {acción}
 ;

- Un (:) separa el lado izq. del derecho de la regla
- Un (;) termina la regla. Por legibilidad, el (;) se ubica solo en una línea.
- La definición consiste de cero o más nombres de terminales, tales como tokens o caracteres literales, y otros símbolos no terminales.
- Cada definición puede tener una acción asociada, ubicada entre llaves.
- La barra vertical (|) permite definiciones alternativas dentro de una regla.
- Los nombres de no terminales van en minúsculas y los nombres de los tokens en mayúsculas por convención.



Especificación YACC

Sección de Reglas

- ▶ Un carácter literal se encierra entre apóstrofes (comillas simples).
- ▶ La `\` tiene un significado especial, para secuencias escape:

<code>'\n'</code>	newline
<code>'\r'</code>	return
<code>'\''</code>	comilla simple
<code>'\\'</code>	barra invertida
<code>'\t'</code>	tab
<code>'\b'</code>	backspace
<code>'\f'</code>	form feed
<code>'\xxx'</code>	carácter cuyo valor es xxx

Especificación YACC

Sección de Reglas

```
%{  
    import java.io.*;  
    import Lex.Lex;  
}%  
  
%token ID CTE ASIG IF THEN ELSE WHILE  
  
%%  
prog          : asignacion  
               ;  
asignacion    : ID ASIG expresion  
               ;  
expresion     : expresion '+' termino  
               | expresion '-' termino  
               | termino  
               ;  
termino      : termino '*' factor  
               | termino '/' factor  
               | factor  
               ;  
factor       : ID  
               | CTE  
               ;
```

Especificación YACC

Sección de Reglas

- ▶ Cualquier **regla** de la gramática puede tener una **acción** asociada
 - ▶ Una acción es una o más sentencias en el lenguaje de desarrollo.

Especificación YACC

Sección de Reglas

```
%{  
    import java.io.*;  
    import Lex.Lex;  
}%  
  
%token ID CTE ASIG IF THEN ELSE WHILE  
  
%%  
prog          : asignacion  
               ;  
  
asignacion    : ID ASIG expression  
               ;  
  
expression    : expression '+' termino {System.out.println("SUMA");}  
               | expression '-' termino {System.out.println("RESTA");}  
               | termino  
               ;  
  
termino       : termino '*' factor {System.out.println("MULTIPLICACIÓN");}  
               | termino '/' factor {System.out.println("DIVISION");}  
               | factor  
               ;  
  
factor        : ID  
               | CTE  
               ;
```

Especificación YACC

Sección de Reglas

- ▶ Cualquier **regla** de la gramática puede tener una **acción** asociada
 - ▶ Una acción es una o más sentencias en el lenguaje de desarrollo.
 - ▶ Frecuentemente, la acción actúa sobre la información de los tokens contenida en la Tabla de Símbolos.

Tokens – **yylval**

- ▶ Para pasar información del token al parser, el Analizador Léxico debe utilizar una variable externa llamada **yylval**.

Especificación YACC

Sección de Reglas

- ▶ El Analizador Léxico asigna a la variable externa **yylval** la referencia a la entrada en la Tabla de Símbolos donde se almacenó el valor léxico del token.
- ▶ YACC provee una notación que permite acceder a los valores retornados por *yylex*:
 - ▶ La notación posicional, $\$n$, permite acceder a la información del n -ésimo token en la definición de una regla.
 - ▶ La variable $\$ \$$ permite asignar información al lado izquierdo de una regla.

Tokens – **yylval**

(YACC para Java)

- YACC genera la clase pública ParserVal

```
public class ParserVal
{
    public int ival;
    public double dval;
    public String sval;
    public Object obj;
    public ParserVal(int val)
    {
        ival=val;
    }
    public ParserVal(double val)
    {
        dval=val;
    }
    public ParserVal(String val)
    {
        sval=val;
    }
    public ParserVal(Object val)
    {
        obj=val;
    }
} //end class
```

Tokens – yylval

(Yacc para Java)

- ▶ Desde el Analizador Léxico, se podrá asignar la variable `yylval` usando:

```
yylval = new ParserVal(stringval); // lexema
yylval = new ParserVal(doubleval);
yylval = new ParserVal(integerval);
...
o:
yylval = new ParserVal(new myTypeOfObject());
```

- ▶ Desde el Analizador Sintáctico, se podrá usar el valor asignado:

```
acceder_TS($1.sval)
$$.ival = $1.ival + $2.ival;
$$dval = $1.dval - $2.dval;
```

Tokens – **yylval**

(Yacc para C)

- ▶ En C, el tipo de **yylval** es int, por defecto.
- ▶ Se puede cambiar el tipo de **yylval** o, definir una unión de tipos de datos múltiples para **yylval**:

```
% union {  
    int entero;  
    char *cadena;  
}
```

Tokens – yylval

(Yacc para C)

- ▶ Desde el Analizador Léxico, se podrá asignar la variable `yylval` usando:
 - ▶ Si se utiliza el tipo `int`, asignado por defecto a `yylval`:
 - ▶ **`yylval = interval;`**
 - ▶ Si se define una unión de tipos de datos múltiples:
 - ▶ **`yylval.entero = interval;`**
 - o
 - ▶ **`yylval.cadena = lexema;`**

- ▶ Desde el Analizador Sintáctico, se podrá usar el valor asignado:
 - ▶ Si se utiliza el tipo `int`, asignado por defecto a `yylval`:
 - ▶ **`$$ = $1;`**
 - ▶ Si se define una unión de tipos de datos múltiples:
 - ▶ **`Acceder_TS($1.cadena);`**
 - o
 - ▶ **`$.entero = $1.entero + $2.entero;`**

Acciones de las reglas

```
expr      : ID          { $$ = $1; }  
          | '(' expr ')' { System.out.println("(" + recuperar_lexema($2) + ")"); }  
          ;
```

- ▶ La primera acción es invocada cuando se reconoce "ID", y retorna la información del token **ID** como el valor de la regla expr.
Esta es la acción por defecto, y puede ser omitida opcionalmente.
- ▶ La segunda acción imprime la expresión completa.
recuperar_lexema accede a la tabla de símbolos para recuperar el lexema correspondiente al identificador reconocido en la primera regla.
- ▶ Si la entrada es: "(x)", la acción de la segunda regla imprimirá "(x)"



Especificación YACC

Sección de Reglas

Acciones

- ▶ El parser generado por YACC guarda los valores de cada token en una variable de trabajo (yyval del mismo tipo que yylval).
- ▶ Las variables de trabajo están disponibles para ser usadas dentro de las acciones de las reglas, y son rotuladas \$1, \$2, \$3, etc.
- ▶ La pseudo-variable \$\$ es el valor a ser retornado por esa invocación de la regla.
- ▶ En el código real, son reemplazadas con las referencias Yacc correctas.
- ▶ Se pueden ejecutar acciones después de cualquier elemento de un conjunto de tokens (no sólo al final), *aunque no se recomienda*.

Especificación YACC

Sección de Reglas

Token error

- ▶ Se puede usar en las reglas un símbolo llamado **error**.
- ▶ No existe una regla que lo defina, ni se incluye en la declaración de tokens.
- ▶ Es un token definido especialmente por Yacc que significa que cualquier entrada que no aparece ninguna de las otras reglas, apareará la que contiene error.
- ▶ Conviene utilizarlo con otro token, que sirve de carácter de *sincronización*. A partir del token erróneo, Yacc descartará tokens hasta encontrar ese carácter (por ej. un `;`).
- ▶ Permite recuperarse de un error, y continuar con la compilación.
- ▶ Se puede asociar una acción que permita informar que hubo un error, y toda la información que se desee agregar.

Especificación YACC

Sección de Reglas

Token error

- El analizador sintáctico podrá seguir compilando aún cuando el código fuente contenga errores.

```
%token ID CTE ASIG IF THEN ELSE WHILE

%%
prog          : asignacion
               ;

asignacion    : ID ASIG expresion ';'
               | error ';' ;

expresion     : expresion '+' termino
               | expresion '-' termino
```

Informar que hubo
error en asignación

Especificación YACC

Sección de Reglas

Gramática de error

- ▶ Se incluirán reglas que describan errores.
- ▶ Para el analizador sintáctico, un código fuente con esos errores será un código válido.
- ▶ ¡Pero no para nuestro compilador!

```
expresion      : expresion '+' termino  
                | expresion '+' '*' termino  
                | expresion '-' termino  
                | termino  
                ;  
  
termino        : termino '*' factor
```

Informar que hubo
error en la expresión

Especificación YACC

Sección de Reglas: Reporte de errores

```
%token ID CTE ASIG IF THEN ELSE WHILE

%%
prog          : asignacion          {System.out.println("Se reconoció un programa");}
               ;

asignacion     : ID ASIG expresion ';' {System.out.println("Asignación");}
               | error ';'           {System.out.println("Error en asignación");}
               ;

expresion      : expresion '+' termino
               | expresion '-' termino
               | termino
               ;

termino        : termino '*' factor
               | termino '/' factor
               | factor
               ;

factor         : ID
               | CTE
               ;
```

Especificación YACC

Sección de Reglas: Reporte de errores

```
%token ID CTE ASIG IF THEN ELSE WHILE
```

```
%%
```

```
prog          : asignacion                {System.out.println("Se reconoció un programa");}
               ;

asignacion     : ID ASIG expresion ';'      {System.out.println("Asignación");}
               | error ';'                 {System.out.println("Error en asignación");}
               ;

expresion      : expresion '+' termino
               | expresion '+' '*' termino {System.out.println("Error: Dos operadores juntos");}
               | expresion '-' termino
               | termino
               ;

termino        : termino '*' factor
               | termino '/' factor
               | factor
               ;

factor         : ID
               | CTE
               ;
```

Especificación YACC

Sección de declaraciones

- ▶ **% union** Declara múltiples tipos de datos para los atributos de los tokens (yyval)

Ejemplo:

```
% union {  
    int entero;  
    char *cadena;  
}
```

- ▶ **% token** Declara los nombres de los tokens.

Si se usa union la sintaxis es:

`% token <elem. de la union que corresponde a este grupo de tokens > lista de tokens`

Especificación YACC

Sección de declaraciones

▶ **% left** Define operadores asociativos a izquierda

▶ **% right** Define operadores asociativos a derecha

El orden de estas declaraciones indica la precedencia (de menor a mayor)
Si se utiliza esta declaración, no es necesario declarar los operadores como tokens.

▶ **% nonassoc** Define operadores no asociativos

▶ **% type** Declara el tipo de los no terminales, cuando se usaron diferentes tipos, y en las acciones asociadas a las reglas, se hacen asignaciones a \$\$

▶ **% start** Declara el símbolo de start. Por defecto es la primera regla

▶ **% prec** Asigna precedencia a una regla



Especificación YACC

Sección de declaraciones

- ▶ Puede contener código en el lenguaje de desarrollo para declarar variables, tipos, etc. `%{`
 declaraciones en lenguaje de desarrollo
 `%}`
- ▶ Cualquier cosa entre `%{` y `%}` es copiada directamente al archivo generado por YACC.

```
%{  
    import java.io.*;  
    import Lex.Lex;  
%}  
  
%token ID CTE ASIG IF THEN ELSE WHILE  
  
%%
```

Especificación YACC

Sección de código

- ▶ La sección de código es opcional, pero puede contener cualquier código en el lenguaje de desarrollo provisto por el usuario (incluso el código del Analizador Léxico).

Especificación YACC

Sección de código

```
factor          : ID
                | CTE
                ;

%%

static Lex lex = null;
static Parser par = null;

public static void main (String [] args) {

    System.out.println("Iniciando compilación...");
    lex = new Lex (args[0]);
    par = new Parser (false);
    par.run();
    System.out.println("Fin compilación");
}

int yylex () {
    int token = lex.getToken();
    yylval = new ParserVal(lex.punteroTS);
    return token;
}

void yyerror (String s){
    System.out.println(s);
}
```



Especificación YACC

- ▶ La mínima especificación YACC consiste en una sección de reglas precedidas por una declaración de los tokens usados en la gramática.

Autómata de Pila

- ▶ Los autómatas finitos son suficientes para el Analizador Léxico.
- ▶ Los A.F. no son suficientes para un Analizador Sintáctico, porque son incapaces de recordar el *estado anterior*.
- ▶ YACC genera un autómata de pila.

Autómata de Pila

- ▶ Tiene un número finito de estados, una función de transición, una entrada, y está equipado con una **pila**.
- ▶ La función de transición trabaja sobre el estado actual, el elemento en el tope de la pila, y el token de entrada actual, produciendo un nuevo estado.
- ▶ Permite a YACC reconocer gramáticas LALR (LookAhead Left Recursive) o LR(I).

Autómata de Pila

- ▶ El estado actual es siempre el del tope de la pila.
- ▶ Inicialmente, la pila contiene sólo el estado 0 y no se ha leído ningún token.
- ▶ El autómata tiene 4 acciones disponibles:
 - ▶ **shift,**
 - ▶ **reduce,**
 - ▶ **accept,**
 - ▶ **y error.**

Autómata de Pila

- ▶ De acuerdo con el estado actual, el parser decide si necesita un token para decidir la próxima acción.
- ▶ Si necesita un token y no lo tiene, llama a yylex para obtener el próximo token.
- ▶ La acción puede provocar que se apilen y desapilen estados en la pila, y que el token sea procesado o no.

Autómata de Pila

- ▶ Una acción **shift**, se ejecuta a partir del token leído.
- ▶ Ejemplo: En un estado, puede haber una acción:

IF shift 34

- ▶ Si el token leído es IF, el estado 34 se convertirá en el estado actual (en el tope de la pila).
- ▶ Se pide un nuevo token.

Autómata de Pila

- ▶ Una acción **reduce** evita que la pila crezca sin límites.
- ▶ Se ejecuta cuando el parser ha visto el lado derecho de una regla y está listo para reemplazar el lado derecho por el izquierdo.
- ▶ Las acciones *reduce* son asociadas con reglas de la gramática.

La acción:

. reduce l8

se refiere a la regla l8 de la gramática.

- ▶ Hace que se desapilen tantos estados como símbolos tenga la regla del lado derecho.

Autómata de Pila

- ▶ Al desapilar estados, luego de un **reduce**, queda descubierto el estado en que el parser estaba al comenzar a procesar la regla.
- ▶ Con este estado y el símbolo del lado izquierdo de la regla, se ejecuta un shift de un nuevo estado a la pila, pero sin leer un nuevo token.
- ▶ Esta acción se llama **goto**.
- ▶ Ejemplo:

A goto 20

A es el lado izquierdo de la regla después del reduce, y 20 será el nuevo estado actual.

Autómata de Pila

- ▶ La acción **accept** indica que el parser ha visto la entrada completa y que ésta cumple con la especificación.

Esta acción aparece sólo cuando el token leído es la marca de fin de archivo.

- ▶ La acción **error** representa un lugar donde el parser no puede continuar el proceso de acuerdo con la especificación.

Ejemplo: Especificación YACC

%token A B C

%%

lista : inicio fin

;

inicio : A B

;

fin : C



Ejemplo: Autómata

state 0

\$accept : _lista \$end

A shift 3
. error

lista goto 1
inicio goto 2

state 1

\$accept : lista_\$end

\$end accept
. error

state 2

lista : inicio_fin

C shift 5
. error

fin goto 4

state 3

inicio : A_B

B shift 6
. error

state 4

lista : inicio fin_ (1)

. reduce 1

state 5

fin : C_ (3)

. reduce 3

state 6

inicio : A B_ (2)

. reduce 2

Autómata de Pila

- ▶ Para obtener el autómata, se debe ejecutar yacc con la opción `-v`

`yacc -J -v gramatica.y`

(byacc para Java)

`byacc -v gramatica.y`

(byacc para C)



y.output

Conflictos en gramáticas Yacc

- ▶ Para la regla:

```
sent : IF '(' cond ')' sent
      | IF '(' cond ')' sent ELSE sent
      ;
```

- ▶ Con la entrada:

```
IF ( C1 ) IF ( C2 ) S1 ELSE S2
```

- ▶ El parser puede reconocer:

```
IF ( C1 )
{
    IF ( C2 )
        S1
}
ELSE
    S2
```

(Reduce)

o

```
IF ( C1 )
{
    IF ( C2 )
        S1
    ELSE
        S2
}
```

(Shift)

Conflictos en gramáticas Yacc

```
%token IF ELSE C
```

```
%%
```

```
sent      : IF '(' cond ')' sent  
          | IF '(' cond ')' sent ELSE sent  
          ;
```

```
cond      : C  
          ;
```

Administrador: Símbolo del sistema

```
D:\Dropbox-hotmail\Dropbox\UNICEN-Tandil\Compiladores I\Herramientas\LEX_YACC\by  
acc para java\Pruebas>yacc -J -v gramatica_conflicto_shift_reduce.y  
yacc: 1 shift/reduce conflict.
```

Conflictos en gramáticas Yacc y.output

```
state 6
  sent : IF '(' cond ')' . sent (1)
  sent : IF '(' cond ')' . sent ELSE sent (2)

  IF shift 1
  . error

  sent goto 7
```

```
7: shift/reduce conflict (shift 8, reduce 1) on ELSE
state 7
  sent : IF '(' cond ')' sent . (1)
  sent : IF '(' cond ')' sent . ELSE sent (2)

  ELSE shift 8
  $end reduce 1
```

```
state 8
  sent : IF '(' cond ')' sent ELSE . sent (2)

  IF shift 1
  . error

  sent goto 9
```

Conflictos en gramáticas Yacc

```
%%
prog          : asignacion
               ;

asignacion     : ID ASIG expresion
               ;

expresion      : expresion '+' termino
               | expresion '-' termino
               | termino
               | factor
               ;

termino        : termino '*' factor
               | termino '/' factor
               | factor
               ;

factor
```

Administrador: Símbolo del sistema

```
D:\Dropbox-hotmail\Dropbox\UNICEN-Tandil\Compiladores I\Herramientas\LEX_YACC\by
acc para java\Pruebas>yacc -J -v gram_conflicto_reduce_reduce.y
yacc: 3 reduce/reduce conflicts.
```

Conflictos en gramáticas Yacc y.output

```
9: reduce/reduce conflict (reduce 6, reduce 9) on $end
9: reduce/reduce conflict (reduce 6, reduce 9) on '+'
9: reduce/reduce conflict (reduce 6, reduce 9) on '-'
state 9
    expresion : factor . (6)
    termino : factor . (9)

$end  reduce 6
'+'   reduce 6
'-'   reduce 6
'*'   reduce 9
'/'   reduce 9
```

Conflictos en gramáticas Yacc

- ▶ Ante la regla

$\text{expr} : \text{expr} \text{ '-' } \text{expr}$

- ▶ La entrada:

$\text{expr} - \text{expr} - \text{expr}$

Puede ser reconocida como:

$(\text{expr} - \text{expr}) - \text{expr}$ (Asociatividad a izquierda)

reduce

○

$\text{expr} - (\text{expr} - \text{expr})$ (Asociatividad a derecha)

shift

Conflictos en gramáticas Yacc

► Soluciones:

- **REESCRIBIR LA GRAMÁTICA**
- Modificar la precedencia / asociatividad de los operadores
- Modificar la precedencia de las reglas
- Reglas de desambiguación de la herramienta

Solución de conflictos: Asociatividad / Precedencia

%left '+' '-'

%left '*' '/'

%%

...

asig : ID '=' expr

expr : expr '+' expr

| expr '-' expr

| expr '*' expr

| expr '/' expr

| '-' expr %prec '*'

| ID

;



Conflictos en gramáticas Yacc

- ▶ Si ni el símbolo de entrada ni la regla tienen precedencia y asociatividad:
 - ▶ En un conflicto **shift-reduce**, la acción por defecto es el shift
 - ▶ En un conflicto **reduce-reduce**, el defecto es reducir por la primera regla (en la especificación Yacc)

Reglas de desambiguación

- ▶ Se consideran las precedencias y asociatividades para aquellos tokens y literales que las tengan.
- ▶ Cuando hay un conflicto **reduce-reduce** o un conflicto **shift-reduce** y, ni el símbolo de entrada ni la regla tienen precedencia y asociatividad, entonces se usan las dos reglas de desambiguación descritas anteriormente, y los conflictos son reportados.
- ▶ Si hay un conflicto **shift-reduce**, y tanto la regla de la gramática como el carácter de entrada tienen precedencia y asociatividad asociadas con ellos, el conflicto se resuelve en favor de la acción (**shift** o **reduce**) asociada con la precedencia más alta. Si las precedencias son iguales, se usa la asociatividad.
 - ▶ La asociatividad a izquierda implica **reduce**;
 - ▶ La asociatividad a derecha implica **shift**;
 - ▶ La no asociatividad implica **error**.

Conflictos en gramáticas Yacc

► REESCRIBIR LA GRAMÁTICA

¿Preguntas?

